

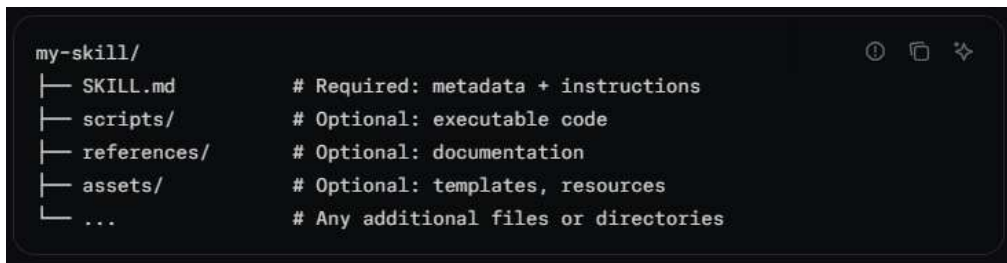
Agent Skills w/ Anthropic

Introduction

- Skills give Claude Code and other agents new abilities to carry out tasks.
- Skills are folders of instructions that extend your agent's capabilities with specialized knowledge.
- Any skill should include a SKILL.md markdown file, which contains the skill's name, description, and main instructions. The main instructions can also refer to other files such as scripts, additional markdown files, and assets such as templates and images.
- Skills are progressively disclosed to the agent, which means that the skill's name and description always live in your agent's context window, but the agent does not load the rest of the instructions into its context until a user request matches the skill's description.
- To use a skill, your agent needs a basic set of tools, filesystem access to read and write files and a bash tool to execute code.
- Your agent can combine skills with MCP and sub-agents to create powerful, agentic workflows.

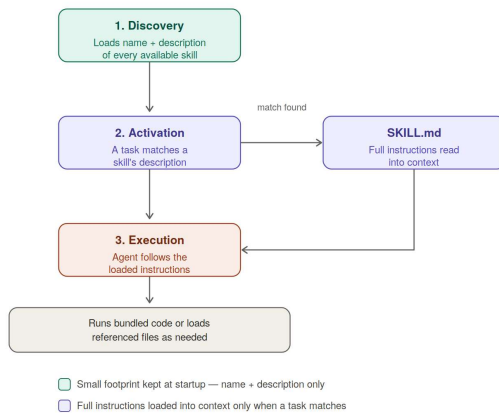
Eg: it can use MCP to get data from external sources, then rely on a skill to know what to do with that data or how to retrieve it efficiently. It can also delegate tasks to a sub-agent with isolated context, which can itself use skills for specialized knowledge.

- So, how do you know when to use a skill? - A workflow that you repeatedly need to ask your agent to implement. Instead of explaining the same workflow every time, you can package it as a skill so your agent automatically knows what to do.
- At its core, a skill is a folder containing a SKILL.md file. This file includes metadata (name and description, at minimum) and instructions that tell an agent how to perform a specific task. Skills can also bundle scripts, reference materials, templates, and other resources.



- Why Agent Skills? - Agents are increasingly capable, but often don't have the context they need to do real work reliably. Skills solve this by packaging procedural knowledge and company-, team-, and user-specific context into portable, version-controlled folders that agents load on demand. This gives agents:
 - **Domain expertise:** Capture specialized knowledge — from legal review processes to data analysis pipelines to presentation formatting — as reusable instructions and resources.
 - **Repeatable workflows:** Turn multi-step tasks into consistent, auditable procedures.
 - **Cross-product reuse:** Build a skill once and use it across any skills-compatible agent.

How agent skills work: progressive disclosure



Why Use Skills

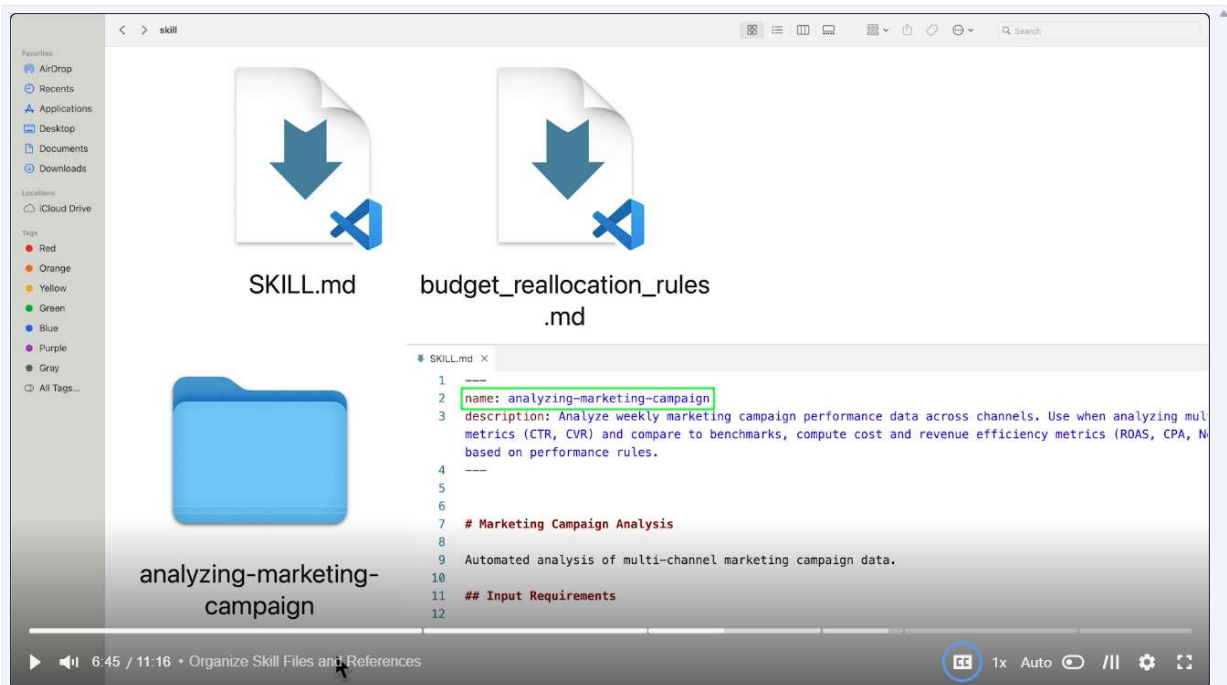
- Skills can be used to package a prompt / set of prompts which need to be used regularly so that instead of going through the whole set of prompts repeatedly and filling session context with things that may not be needed, we can reference the skill (with its SKILL.md containing the prompt/set of prompts, rules and other references which may only be read conditionally, making optimum usage of context window)

In example shown below - marketing data analysis is carried out through the SKILL.md file and only when user asks about Budget Reallocation, is the budget reallocation rules markdown referred to (created based on other set of prompts used for another periodic task)

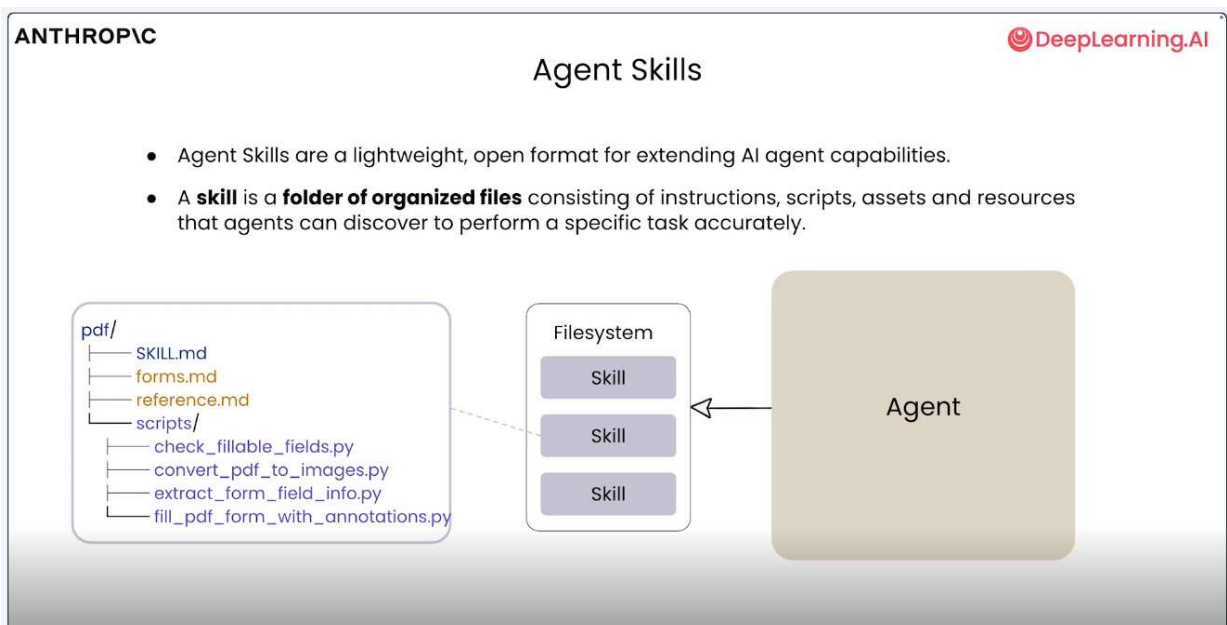
```

1 # Marketing Campaign Analysis
42 ## Efficiency Analysis
54 Compare to user-provided targets. If not provided, use these defaults:
55
56 - **Target ROAS**: 4.0x minimum
57 - **Max CPA**: $50
58
59 ## Output Format
60
61 Present results as tables with status indicators:
62
63 **Funnel Analysis Table:**
64 | Channel | CTR Actual | CTR Benchmark | CVR Actual | CVR Benchmark | CVR Diff |
65
66 **Efficiency Analysis Table:**
67 | Channel | ROAS | Status | CPA | Status | Net Profit | Status |
68
69 Status indicators:
70
71 - ROAS: "[OK] Above" if >= target, "[X] Below" if < target
72 - CPA: "[OK] Below" if <= max, "[X] Above" if > max
73 - Net Profit: "[OK] Positive" if > 0, "[X] Negative" if <= 0
74
75 Follow each table with brief channel-by-channel interpretation highlighting key insights and recommended actions.
76
77 ## Budget Reallocation
78
79 If user asks about budget reallocation, read 'references/budget_reallocation_rules.md' for the complete decision framework including eligibility rules,
    performance-based actions, and constraints.
    
```

- Every skill made needs to have a name (to link/refer to the skill) and description (to know when the skill is to be used)- this needs to be mentioned at the very start of the SKILL.md file in yaml format. This becomes the skill metadata which is always loaded into the context window when a skill gets added/loaded from file system.
- Structure: Folder (named after the skill) - contains SKILL.md (which has prompt/tasks/rules) and references folder for external references. Any references that SKILL.md makes to should be maintained in this folder - zipped folder uploaded to Claude as skill for future use.
- The SKILL.md is loaded only when the skill is triggered and only depending on what is required, additional resources (like reference markdowns, scripts, etc.) will loaded as needed. - This is progressive disclosure
- Naming conventions for folder: use lower case letters, dashes between words and don't use reserved keywords like Claude, Anthropic, etc.

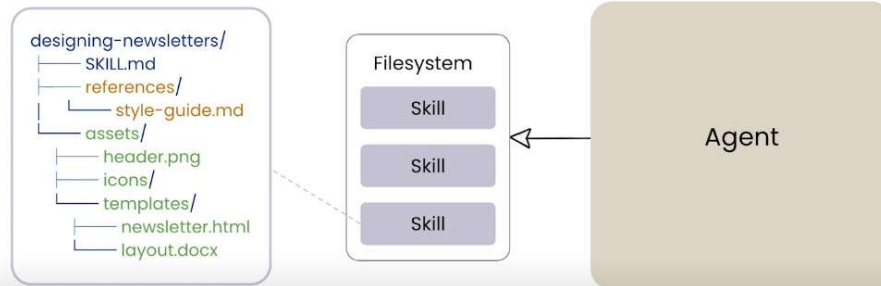


- As skills are an open standard, this is supported in other coding environments like Codex, Gemini CLI, and much more.



Agent Skills

- Agent Skills are a lightweight, open format for extending AI agent capabilities.
- A **skill** is a **folder of organized files** consisting of instructions, scripts, assets and resources that agents can discover to perform a specific task accurately.



- **Old way vs. new way of building agents:** Teams used to build separate, single-purpose agents for coding, research, finance, etc., each with its own hardcoded tools and context. Turns out most agents only need the same simple scaffolding underneath (basic tools like bash and a filesystem to find, edit, and run things) — the domain-specific part was really just knowledge, not architecture.
- **Skills fill the gap that generic agents lack:** A bare-bones agent is easy to build and scale, but it doesn't know *how* to do finance analysis "the right way" or follow a specific research process. Skills solve this by loading that expertise (procedural knowledge and company, team and user-specific context) on demand, giving the agent:
 - **repeatable workflows** (clear steps so the output is predictable, not random each time), and
 - **new capabilities** (things Claude couldn't do at all before, like handling a new data format) — all without bloating the agent's core setup.

Why Agent Skills?

Use Cases

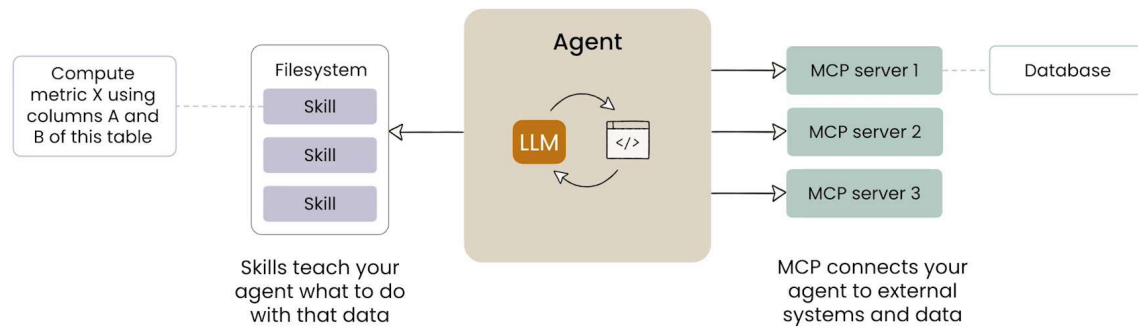
- | | |
|--|---|
| <div style="background-color: #d9d9e3; padding: 10px; margin-bottom: 10px; text-align: center;">Domain Expertise</div> <div style="background-color: #d9d9e3; padding: 10px; margin-bottom: 10px; text-align: center;">Repeatable Workflow</div> <div style="background-color: #d9d9e3; padding: 10px; text-align: center;">New Capabilities</div> | <ul style="list-style-type: none"> • Brand guidelines and templates • Legal review processes • Data analysis methodologies
<ul style="list-style-type: none"> • Weekly marketing campaign review • Customer call prep workflow • Quarterly business review
<ul style="list-style-type: none"> • Creating presentations • Generating Excel sheets or PDF reports • Building MCP servers |
|--|---|

Without Skills

- Describe your instructions and requirements every time
- Bundle all your references and supporting files every time
- Ensure the workflow or outputs are always consistent

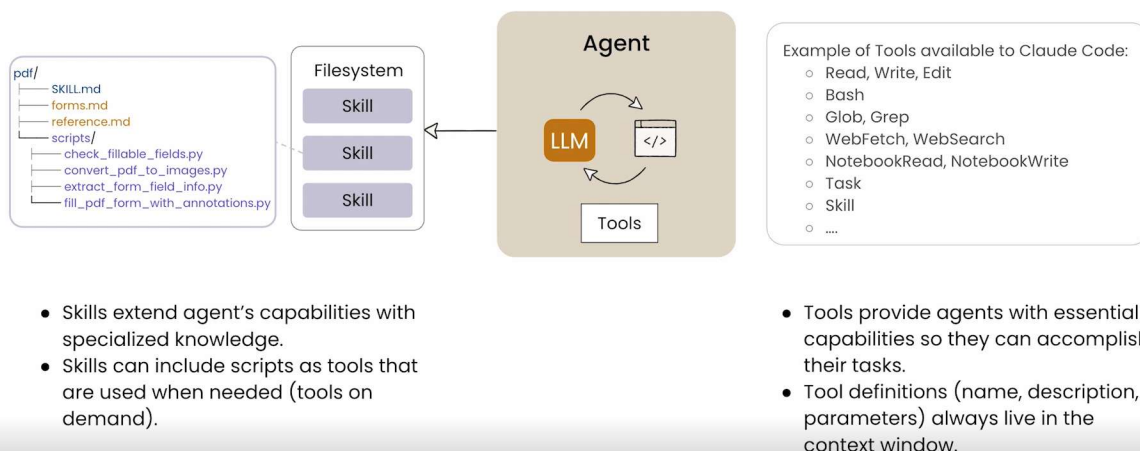
- Simple takeaway: skills aren't just standalone add-ons — you can **stack multiple skills together** in one workflow. By referencing which skills apply and the order of steps needed, you turn an otherwise unpredictable (non-deterministic) system into one that produces **consistent, predictable results** for complex, multi-part tasks.

Skills vs MCP



- **MCP (Model Context Protocol)** = the connection to outside tools and data. It's what lets your agent reach external systems it wouldn't otherwise know about — a database, Google Drive, or any other data source — essentially plugging in the raw tools and information.
- **Skills** = the instructions for what to do with those tools. MCP brings the "ingredients" (tools + data) to the table; skills provide the "recipe" — the repeatable steps that tell the agent how to combine and use that external data to reliably produce the output you actually want (a calculation, a report, a specific metric).
- **Skills v/s MCPs:** MCP gets your agent *access* to outside stuff; skills tell it *how* to use that access in a consistent, repeatable way.

Skills vs Tools



Skills v/s Tools:

- **Tools are the low-level building blocks; skills are the know-how for using them.** Think hammer, saw, and nails (tools) versus knowing how to build a bookshelf (skill). Tools are the underlying capabilities that let an agent access systems, execute code, or read/write files — and in fact, tools are what power the ability to even generate, read, and load skills in the first place.
- **Skills sit on top of tools and add specialized knowledge.** A skill can pull in extra files and scripts that need to run, but the actual *execution* of those scripts and the loading of those files/folders is still handled by tools underneath. Tools themselves can come from three places: built into the agentic ecosystem, custom-written, or loaded via the model context protocol (MCP).
- **The key structural difference is context cost and when each loads.** Tool definitions live in the context window at all times, while skills are progressively loaded only when needed. This is what makes them work well together: a skill can create a predictable workflow and pull in a script "like a tool" on demand — so if a tool isn't needed in every conversation, it can be

tucked inside a skill and only loaded through progressive disclosure when the task actually calls for it.

- **Can skills restrict tool usage?** - Yes. A SKILL.md file's frontmatter can declare something like an allowed-tools list, which scopes what the agent is permitted to use while that skill is active. This matters for exactly the "predictability" theme from the transcript: if a skill is meant to, say, only read and reformat a spreadsheet, restricting it to file-read/write tools (and denying network or shell access) makes its behavior more constrained and auditable — you're not just adding capability, you're also fencing it in. So skills act as both a capability unlock (bring in scripts/knowledge) and, optionally, a capability limiter (restrict which tools apply while following that skill's instructions).
- Earlier, tool definitions (a full JSON schema: name, description, and the exact parameter structure the model needs to format a valid call) had to sit in context the whole conversation, while Skill was triggered only on the basis of matching a task to a plan-language description. So its front-loaded footprint can be tiny (just name + one-line description), with the heavy content (the actual instructions) deferred until matched. However, some environments (including current versions of Claude AI) have started applying that same progressive-disclosure trick to tools too — a tool can be marked "deferred," where only its name is visible at first, and its full schema gets pulled in on demand via a search call before it's usable.

Subagents

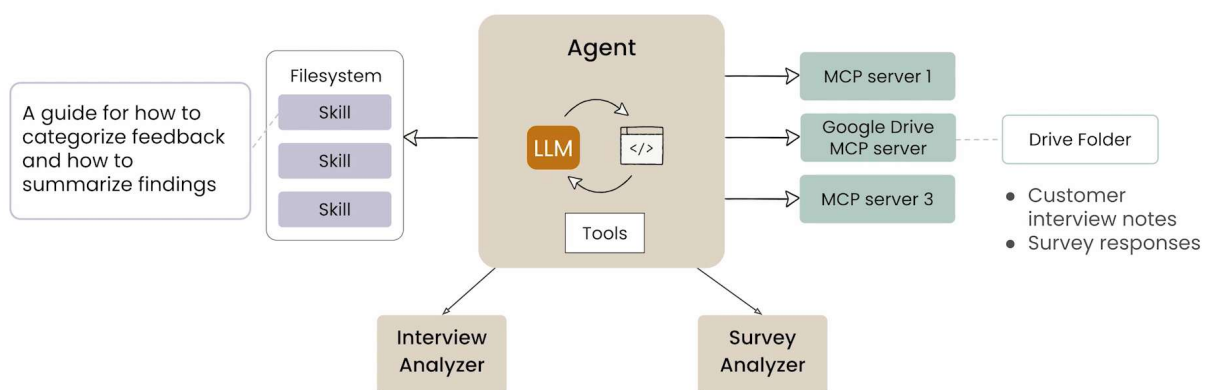
- **A subagent is a child process spawned by a main (orchestrator) agent.** It runs with its own isolated context, gets fine-grained/limited tool permissions, can execute in parallel with other subagents, and reports its results back to the parent. Subagents can come from ecosystems like Claude Code or the Agent SDK, or be custom-built.
- **Subagents can be scoped to specific skills, just like the main agent.** The main agent orchestrates and can pull in whatever skills it needs, but each subagent can also be restricted to only the skills relevant to its job — so permissions and knowledge access are controlled at both levels (main agent and subagent).
- **This combination is what makes subagents genuinely useful in practice.** Example given: a "code reviewer" subagent whose entire job is analyzing a codebase — it leverages a skill that encodes exactly how your team or company wants code reviews done. So the subagent supplies the isolated, parallel execution context, and the skill supplies the specific procedural knowledge for that narrow task.

Use Case:

ANTHROPIC

DeepLearning.AI

Customer Insight Analyzer



Summary:

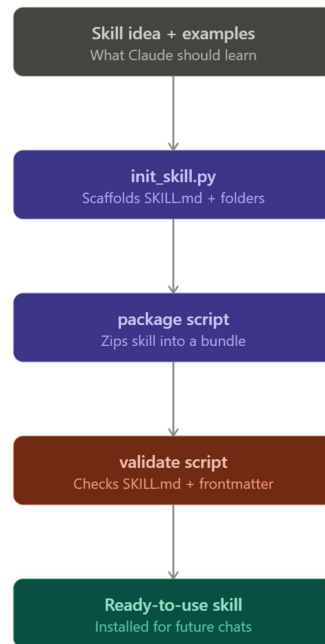
Summary

Feature	Skills	Prompts	Subagents	MCP
What it provides	Procedural knowledge	Moment-to-moment instructions	Task delegation	Tool connectivity
Persistence	Across conversations	Single conversation	Across sessions	Continuous connection
Contains	Instructions + code + assets	Natural language	Full agent logic	Tool definitions
When it loads	Dynamically, as needed	Each turn	When invoked	Always available
Best for	Specialized expertise	Quick requests	Specialized tasks	Data access

- Prompts are the atomic unit — how you talk to a model — but they don't scale across teams or companies on their own. Skills solve that by bundling prompts, conversations, code, and assets into something reusable. Subagents can then use those skills to carry out delegated tasks, and those subagents in turn draw on tools provided by the main agent.
- **The common thread across all of them is being deliberate about the context window.** Treat context as a shared, limited resource: subagents help by keeping heavy work in their own isolated context instead of bloating the main one; MCP loads external data/tools only when connected; skills load their content progressively, only when a task matches. Different mechanisms, same goal — don't stuff everything into context all the time.
- **Where each one's "memory" lives, and when to reach for which.** Subagents can persist state across sessions between themselves and the parent agent; skills persist across conversations with the user/application. As a rule of thumb: **use skills for procedural, predictable, repeatable workflows**, and **reserve subagents for full agentic reasoning**, only when a task genuinely needs specialized, isolated execution.

Claude Skills

- **Two tiers of pre-built skills** - **Document skills** (Excel, Word, PowerPoint, PDF) — built into Claude AI/Desktop, always on, can't be toggled off and **Example skills** (like skill-creator) — live in the same GitHub repo but are off by default in Settings → Capabilities, except skill-creator itself, which ships enabled.
- skill-creator - a skill that helps in building other skills for you



Creating Custom Skills

- Every skill needs a `SKILL.md` with required YAML frontmatter (`name`, `description` — both character-capped, name in `verb-ing` lowercase-hyphen form, description stating *what* it does and *when* to use it) plus optional fields (license, compatibility, arbitrary metadata). Beyond the frontmatter there's no format restriction, but best practice is step-by-step instructions, explicit edge cases, under 500 lines, and forward slashes even on Windows. Three optional folders extend it — `scripts/`, `references/`, `assets/` — all loaded only when needed (progressive disclosure again).
- **Freedom vs. determinism** is the key design decision: low freedom (exact sequence, no ambiguity) for best-practice-following tasks; high freedom (multiple valid colors/styles/fonts) for creative output.

Skills with Claude API

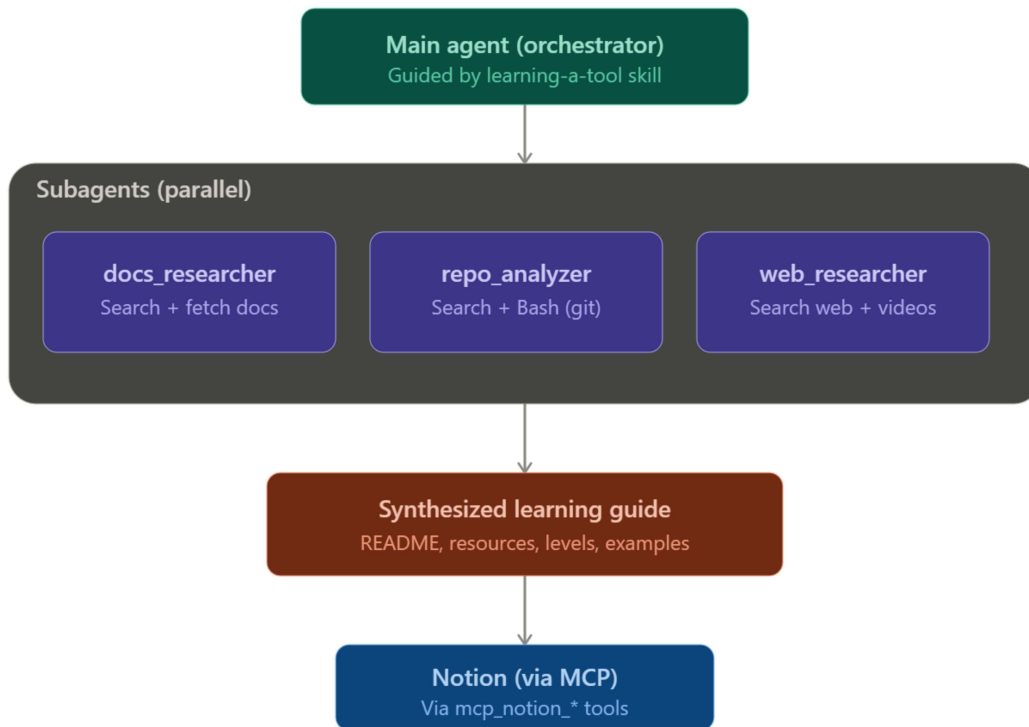
- Two things change versus Claude AI/Desktop:
 - Skills made in Claude AI/Desktop don't carry over to the API or Claude Code — separate environments
 - Claude AI/Desktop gives you a sandboxed container + internet access for free (Settings → Capabilities → "Code execution and file creation"), but on the API you wire that up yourself with the Code Execution Tool (bash + filesystem, sandboxed, no internet, pre-installed libs only) and the Files API (upload inputs, download generated outputs by `file_id`)

-> Anthropic is provisioning a container behind the scenes for you: a small Linux sandbox with its own filesystem where Claude can run bash commands, write and read files, and — importantly — reach the internet, so it can `pip install` a library it doesn't already have, or fetch something from the web mid-task. You never touch any of that plumbing; you just get the capability.

-> **The API is the raw building block, not the finished product.** When you're building your own application against the Messages API, Anthropic doesn't assume you want a sandbox running by default — you must explicitly ask for one by adding the code execution tool to your request. Doing so spins up a genuinely isolated container per session: its own filesystem, its own bash shell, its own CPU/memory/disk allotment, walled off from the host machine and from every other customer's containers. Claude still can't download a new package at runtime; it's stuck with whatever's pre-installed in the container image (pandas, numpy, scipy, matplotlib, `python-docx`, `python-pptx`, `pypdf`, etc.). **Files still move in and out only via the Files API** — upload a file, reference it with a `container_upload` block, and pull generated output back

out by its file ID. That mailbox-in/mailbox-out design hasn't changed.

Example



- A main agent orchestrates three subagents (`docs_researcher`, `repo_analyzer`, `web_researcher`), each with its own scoped tools.
- A single skill — `learning-a-tool` — guides only the *main agent's* workflow: research phase per subagent → organize findings into progressive levels (via a separate "progressive learning" reference file: overview → install → concepts → patterns → next steps) → fixed output structure.
- Getting this running requires explicitly listing every tool any subagent needs under the main agent's `allowed_tools` (subagents inherit nothing by default), plus adding the `Task` tool (to dispatch subagents) and the `Skill` tool (to read skills from `.claude/skills/`).

Skills with the Claude Code

- **CLAUDE.md vs. a skill is a real design decision, not just two file types.** CLAUDE.md is always in context, every conversation. So it's for things true of the whole project (tech stack, architecture, where files live). A skill is for a subset of conventions relevant to one recurring task (e.g., "how we add a CLI command"), loaded only when that task comes up. The rule of thumb given: **if it needs to be everywhere, it goes in CLAUDE.md, if it's only relevant to one repeatable job, make it a skill.**
- **Subagents don't inherit skills from their parent**, one must explicitly assign a skill to each subagent (by name, in its config).
- **The single most important nuance in this transcript:** skills behave differently depending on who's using them. When the *main agent* uses a skill, it's read progressively — `SKILL.md` first, then any referenced files only as needed. When a *subagent* is dispatched with a skill attached, the **entire `SKILL.md` is preloaded upfront** — no progressive disclosure of anything beyond that file. That's a meaningfully different cost/behavior profile worth remembering if you're deciding what to put directly in a subagent's `SKILL.md` vs. what to push into a referenced file.

Skills with the Claude Agent SDK

- Walkthrough video of implementing a multi agent system using Claude SDK (<https://learn.deeplearning.ai/courses/agent...>)
- Agent SDK = exposed as a library you call from Python/TypeScript instead of a CLI.